Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи № 5

« Обробка помилок, файли та серіалізація»

Виконала:

студентка 3 курсу

групи ФЕП-31

Антошкова Аліна Олександрівна

Перевірив:
Доц. Куньо Іван Михайлович

Львів – 2026

Мета: Сформувати практичні навички створення надійних Python-програм, які коректно працюють із механізмом винятків, файловими ресурсами за допомогою модуля pathlib, конфігураційними файлами у форматі YAML, системою логування logging, контекстними менеджерами та структурованими форматами даних (CSV, JSON); опанувати потокове читання (streaming I/O), атомарний запис результатів та принципи проектування надійних систем (EAFP, Fail-fast, Exception chaining).

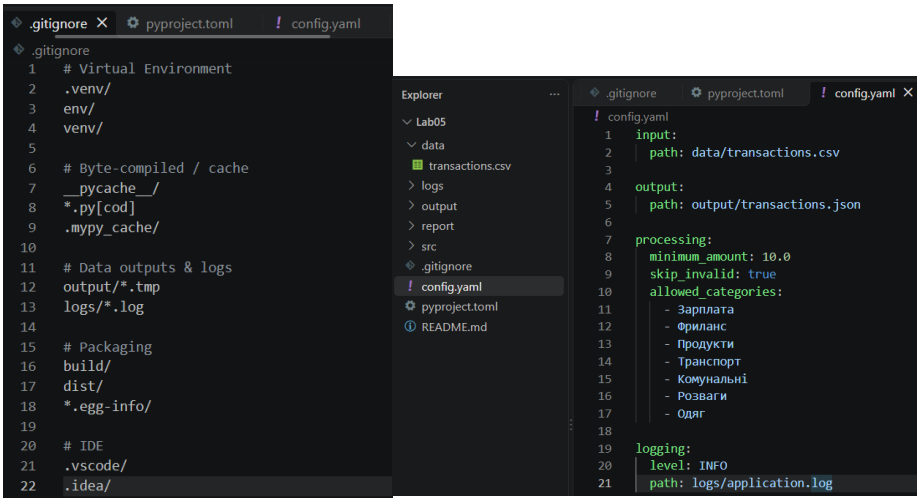
Варіант 12: Імпорт та експорт фінансового трекера.

Постановка завдання:

- Розробити надійний модуль імпорту, валідації та експорту фінансових транзакцій із полями: дата (date), категорія (category), сума (amount), тип операції (type: income/expense) та опис (description).
- Забезпечити зовнішню конфігурацію застосунку через файл config.yaml із визначенням шляхів до ресурсів, порогу мінімальної суми та переліку дозволених категорій.
- Реалізувати потокове читання вхідного CSV-файлу без повного завантаження даних у пам'ять.
- Створити власну ієрархію винятків (ApplicationError, ConfigurationError, RecordValidationError, DataExportError) та застосувати механізм exception chaining (raise ... from error).
- Забезпечити підтримку двох політик надійності: суворої (strict mode), де перша помилка перериває роботу, та толерантної (tolerant mode), де некоректні рядки реєструються в лозі й пропускаються.
- Реалізувати атомарний експорт відфільтрованих валідних даних у формат JSON через тимчасовий .tmp файл.
- Створити власний контекстний менеджер для заміру часу та безпечного логування операцій.
- Перевірити реакцію системи на критичні помилки конфігурації (принцип Fail-fast)

Хід роботи:

1. Додавання базової конфігурації проєкту Lab05 (.gitignore, pyproject.toml, config.yaml, README.md, тестовий CSV з помилками) та виконання початкового коміту



```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython (main)
$ git add Lab05/.gitignore Lab05/pyproject.toml Lab05/config.yaml Lab05/README.m
d Lab05/data/transactions.csv

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython (main)
$ git commit -m "Lab05: initial setup, config.yaml, test data and packaging conf
iguration"
[main 345efa0] Lab05: initial setup, config.yaml, test data and packaging config
uration
 5 files changed, 78 insertions(+)
 create mode 100644 Lab05/.gitignore
 create mode 100644 Lab05/README.md
 create mode 100644 Lab05/config.yaml
 create mode 100644 Lab05/data/transactions.csv
 create mode 100644 Lab05/pyproject.toml

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython (main)
$ git push origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 1.83 KiB | 623.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/AntoskovaA/paython.git
   a7757e9..345efa0  main -> main
```

2. Налаштування віртуального оточення, створення доменної ієрархії винятків (exceptions.py) та типізованих моделей конфігурації й даних (models.py)

```
PS C:\Users\Alina\Desktop\пайтон\paython\Lab05> python -m venv .venv
PS C:\Users\Alina\Desktop\пайтон\paython\Lab05>  .venv/Scripts/activate
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab05> python -m pip install -e .
Obtaining file:///C:/Users/Alina/Desktop/%D0%BF%D0%B0%D0%B9%D1%82%D0%BE%D0%BD/paython/Lab05
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Collecting pyyaml>=6.0 (from reliable-finance==0.1.0)
  Using cached pyyaml-6.0.3-cp312-cp312-win_amd64.whl.metadata (2.4 kB)
Using cached pyyaml-6.0.3-cp312-cp312-win_amd64.whl (154 kB)
Building wheels for collected packages: reliable-finance
  Building editable for reliable-finance (pyproject.toml) ... done
  Created wheel for reliable-finance: filename=reliable_finance-0.1.0-0.editable-py3-none-any.wh
l size=1665 sha256=f8049c2e7fa7a6a9c7ee794af793ec3ae701f5dff589d87cad9006d9978fbbd6
  Stored in directory: C:\Users\Alina\AppData\Local\Temp\pip-ephem-wheel-cache-4fw9j8pk\wheels\e
a\55\0e\a8bb53669077d4143c41c2530998ddf0d2dc3dc6a9ebdb53ad
Successfully built reliable-finance
Installing collected packages: pyyaml, reliable-finance
Successfully installed pyyaml-6.0.3 reliable-finance-0.1.0
```

src > reliable_finance > exceptions.py > DataExportError

```python
class ApplicationError(Exception):
    """Базовий виняток застосунку."""


class ConfigurationError(ApplicationError):
    """Помилка валідації або читання файлу конфігурації."""


class DataError(ApplicationError):
    """Базовий виняток обробки даних."""


class DataImportError(DataError):
    """Помилка відкриття або читання вхідного потоку."""


class RecordValidationError(DataError):
    """Помилка валідації конкретного рядка даних із контекстом."""

    def __init__(
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
    ) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field


class DataExportError(DataError):
    """Помилка запису або серіалізації вихідних даних."""
```

src > reliable_finance > models.py > TransactionRecord > to_dict

```python
from dataclasses import asdict, dataclass
from pathlib import Path


@dataclass(frozen=True, slots=True)
class ProcessingConfig:
    minimum_amount: float
    skip_invalid: bool
    allowed_categories: tuple[str, ...]


@dataclass(frozen=True, slots=True)
class LoggingConfig:
    level: str
    path: Path


@dataclass(frozen=True, slots=True)
class AppConfig:
    input_path: Path
    output_path: Path
    processing: ProcessingConfig
    logging: LoggingConfig


@dataclass(frozen=True, slots=True)
class TransactionRecord:
    date: str
    category: str
    amount: float
    op_type: str
    description: str

    def to_dict(self) -> dict[str, object]:
        return asdict(self)
```

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git add src/reliable_finance/__init__.py src/reliable_finance/exceptions.py src/reliabl
e_finance/models.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git commit -m "Lab05: implement custom exception hierarchy and immutable dataclass mode
ls"
[main ac3c1ea] Lab05: implement custom exception hierarchy and immutable dataclass models
 3 files changed, 68 insertions(+)
 create mode 100644 Lab05/src/reliable_finance/__init__.py
 create mode 100644 Lab05/src/reliable_finance/exceptions.py
 create mode 100644 Lab05/src/reliable_finance/models.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git push origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (8/8), 1.26 KiB | 646.00 KiB/s, done.
Total 8 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/AntoskovaA/paython.git
   345efa0..ac3c1ea  main -> main
```

3. Реалізація завантаження YAML-конфігурації (Fail-fast), системи логування та власного контекстного менеджера logged_operation

models.py | config.py | logging_config.py | README.md | transac

rc > reliable_finance > logging_config.py > configure_logging

```python
import logging
from pathlib import Path


def configure_logging(level: str, path: Path) -> None:
    """Налаштування логера з виводом у файл та консоль."""
    path.parent.mkdir(parents=True, exist_ok=True)
    numeric_level = getattr(logging, level.upper(), logging.INFO)

    logging.basicConfig(
        level=numeric_level,
        format="%(asctime)s | %(levelname)-7s | %(name)s | %(message)s",
        handlers=[
            logging.FileHandler(path, encoding="utf-8"),
            logging.StreamHandler(),
        ],
        force=True,
    )
```

```python
from collections.abc import Generator
from contextlib import contextmanager
import logging
from time import perf_counter

logger = logging.getLogger("reliable_finance.utils")


@contextmanager
def logged_operation(operation_name: str) -> Generator[None, None, None]:
    """Власний контекстний менеджер для фіксації тривалості операції та без
    logger.info(">>> Початок операції: %s", operation_name)
    start_time = perf_counter()
    try:
        yield
    except Exception:
        logger.exception("!!! Збій під час операції: %s", operation_name)
        raise
    else:
        elapsed = perf_counter() - start_time
        logger.info("<<< Операцію '%s' успішно завершено за %.6f с", operat
```

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git add src/reliable_finance/config.py src/reliable_finance/logging_config.py src/relia
ble_finance/utils.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git commit -m "Lab05: implement YAML configuration loader, logging system and context m
anager"
[main 931a344] Lab05: implement YAML configuration loader, logging system and context man
ager
 3 files changed, 83 insertions(+)
 create mode 100644 Lab05/src/reliable_finance/config.py
 create mode 100644 Lab05/src/reliable_finance/logging_config.py
 create mode 100644 Lab05/src/reliable_finance/utils.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git push origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 2.21 KiB | 2.21 MiB/s, done.
Total 8 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/AntoskovaA/paython.git
   ac3c1ea..931a344  main -> main
```

4. Реалізація потокового читання CSV, валідатора з exception chaining, атомарного експорту в JSON та точки входу main.py

```python
from collections.abc import Iterable
import json
from pathlib import Path
from reliable_finance.exceptions import DataExportError
from reliable_finance.models import TransactionRecord


def atomic_export_json(records: Iterable[TransactionRecord], path: Path) -> int:
    """Атомарний потоковий експорт у JSON через тимчасовий файл."""
    path.parent.mkdir(parents=True, exist_ok=True)
    temp_path = path.with_suffix(path.suffix + ".tmp")
    count = 0

    try:
        with temp_path.open("w", encoding="utf-8") as file:
            file.write("[\n")
            first = True
            for record in records:
                if not first:
                    file.write(",\n")
                json.dump(record.to_dict(), file, ensure_ascii=False, indent=2)
                first = False
                count += 1
            file.write("\n]\n")

        # Атомарне заміщення цільового файлу
        temp_path.replace(path)
    except (OSError, TypeError) as error:
        temp_path.unlink(missing_ok=True)
        raise DataExportError(f"Збій експорту у файл {path}: операцію скасовано") from error

    return count
```

```python
import csv
from collections.abc import Iterator
from pathlib import Path
from reliable_finance.exceptions import DataImportError


def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, str]]]:
    """Потокове читання рядків CSV із передачею номера рядка для звітності."""
    try:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            for line_number, row in enumerate(reader, start=2):
                yield line_number, row
    except OSError as error:
        raise DataImportError(f"Неможливо прочитати вхідний файл: {path}")
```

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git add src/reliable_finance/readers.py src/reliable_finance/validators.py src/reliable
_finance/services.py src/reliable_finance/exporters.py src/reliable_finance/main.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git commit -m "Lab05: implement streaming readers, exception chaining, atomic JSON expo
rt and main pipeline"
[main 43483f5] Lab05: implement streaming readers, exception chaining, atomic JSON export
 and main pipeline
 5 files changed, 219 insertions(+)
 create mode 100644 Lab05/src/reliable_finance/exporters.py
 create mode 100644 Lab05/src/reliable_finance/main.py
 create mode 100644 Lab05/src/reliable_finance/readers.py
 create mode 100644 Lab05/src/reliable_finance/services.py
 create mode 100644 Lab05/src/reliable_finance/validators.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git push origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 3.96 KiB | 1.98 MiB/s, done.
Total 10 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/AntoskovaA/paython.git
   931a344..43483f5  main -> main
```

5. Демонстрація роботи застосунку в толерантному режимі (tolerant mode): реєстрація некоректних записів у лозі та збереження валідних транзакцій

```
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab05> python -m reliable_finance.main
2026-10-19 15:17:51,767 | INFO    | reliable_finance.utils | >>> Початок операції: Імпорт, валідація та експорт фінансових транзакцій
2026-10-19 15:17:51,769 | WARNING | reliable_finance.service | Некоректний запис у рядку 4 (поле 'category'): Категорія не вказана
2026-10-19 15:17:51,769 | WARNING | reliable_finance.service | Некоректний запис у рядку 6 (поле 'category'): Категорія 'Казино' не входить до переліку дозволених у конфігурації
2026-10-19 15:17:51,775 | WARNING | reliable_finance.service | Некоректний запис у рядку 7 (поле 'amount'): Сума має бути строго більшою за 0 (отримано -1200.0)
2026-10-19 15:17:51,776 | WARNING | reliable_finance.service | Некоректний запис у рядку 9 (поле 'type'): Некоректний тип операції: 'invalid_type'. Очікується income або expense
2026-10-19 15:17:51,780 | INFO    | reliable_finance.utils | <<< Операцію 'Імпорт, валідація та експорт фінансових транзакцій' успішно завершено за 0.011597 с
2026-10-19 15:17:51,781 | INFO    | reliable_finance.main | Статистика обробки: Всього=9 | Валідних=5 | Некоректних=4 | Експортовано=5
```

6. Результат успішної атомарної серіалізації: чистий JSON-файл у каталозі output/

output > {} transactions.json > ...


```json
[
  {
    "date": "2026-10-01",
    "category": "Зарплата",
    "amount": 32000.0,
    "op_type": "income",
    "description": "Основна зарплата"
  },
  {
    "date": "2026-10-02",
    "category": "Продукти",
    "amount": 1450.5,
    "op_type": "expense",
    "description": "Супермаркет"
  },
  {
    "date": "2026-10-04",
    "category": "Транспорт",
    "amount": 280.0,
    "op_type": "expense",
    "description": "Проїзд у транспорті"
  },
  {
    "date": "2026-10-07",
    "category": "Комунальні",
    "amount": 2100.0,
    "op_type": "expense",
    "description": "Оплата компослуг"
  },
  {
    "date": "2026-10-09",
    "category": "Одяг",
    "amount": 1900.0,
    "op_type": "expense",
    "description": "Купівля куртки"
  }
]
```


7. Перевірка строгого режиму (strict mode): аварійна зупинка конвеєра при виявленні першого некоректного запису з повним трасуванням помилки

main.py | config.yaml

config.yaml

```yaml
input:
  path: data/transactions.csv

output:
  path: output/transactions.json

processing:
  minimum_amount: 10.0
  skip_invalid: false
  allowed_categories:
    - Зарплата
    - Фриланс
    - Продукти
    - Транспорт
    - Комунальні
    - Розваги
    - Одяг

logging:
  level: INFO
  path: logs/application.log
```

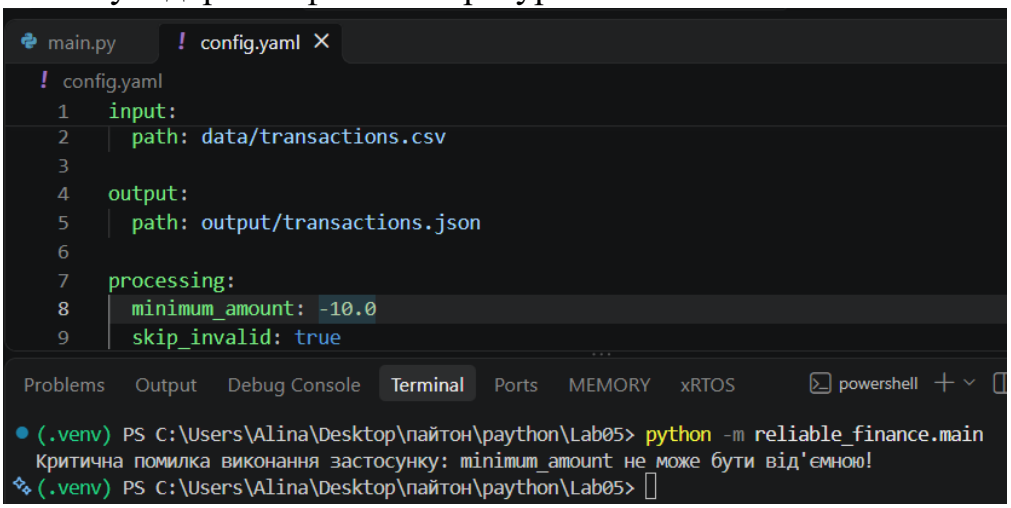
Для перевірки стійкості конвеєра було використано тестовий файл data/transactions.csv, що містить 9 записів (5 коректних та 4 рядки з навмисними помилками валідації: порожня категорія, від'ємна сума, недозволена категорія, неіснуючий тип операції).

Результати роботи програми в залежності від конфігураційного параметра skip_invalid:

Параметр порівняння	Strict Mode (`skip_invalid: false`)	Tolerant Mode (`skip_invalid: true`)
Кількість опрацьованих рядків	3 (перервано на рядку № 4) DOCX	9 (весь вхідний датасет) DOCX
Кількість валідних записів	2	5
Кількість некоректних записів	1 (критичний збій)	4 (зафіксовано у лозі як WARNING) DOCX
Кількість збережених у JSON	0 (операцію скасовано, `.tmp` видалено)	5 (експортовано до `output/transactions.json`) DOCX
Код та статус завершення	`SystemExit(1)` (аварійна зупинка) DOCX	`Код 0` (успішне завершення)
Основна сфера застосування	Фінансові транзакції, де неприпустима втрата жодного запису DOCX	Обробка логів, телеметрії чи аналітики, де допустимий пропуск пошкоджених записів DOCX

```
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab05> python -m reliable_finance.main
2026-10-19 15:22:02,317 | INFO     | reliable_finance.utils | >>> Початок операції: Імпорт, валідація та е
кспорт фінансових транзакцій
2026-10-19 15:22:02,319 | WARNING | reliable_finance.service | Некоректний запис у рядку 4 (поле 'categor
y'): Категорія не вказана
2026-10-19 15:22:02,319 | ERROR    | reliable_finance.utils | !!! Збій під час операції: Імпорт, валідаці
 та експорт фінансових транзакцій
Traceback (most recent call last):
  File "C:\Users\Alina\Desktop\пайтон\paython\Lab05\src\reliable_finance\utils.py", line 15, in logged_op
eration
    yield
  File "C:\Users\Alina\Desktop\пайтон\paython\Lab05\src\reliable_finance\main.py", line 40, in main
    exported_count = atomic_export_json(filtered, config.output_path)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Alina\Desktop\пайтон\paython\Lab05\src\reliable_finance\exporters.py", line 18, in atomi
c_export_json
    for record in records:
                  ^^^^^^^
  File "C:\Users\Alina\Desktop\пайтон\paython\Lab05\src\reliable_finance\services.py", line 55, in filter
_by_minimum_amount
    for record in records:
                  ^^^^^^^
  File "C:\Users\Alina\Desktop\пайтон\paython\Lab05\src\reliable_finance\services.py", line 31, in import
_transactions
    record = parse_transaction(
             ^^^^^^^^^^^^^^^^^^
  File "C:\Users\Alina\Desktop\пайтон\paython\Lab05\src\reliable_finance\validators.py", line 18, in pars
e_transaction
    raise RecordValidationError("Категорія не вказана", line_number=line_number, field="category")
reliable_finance.exceptions.RecordValidationError: Категорія не вказана
2026-10-19 15:22:02,324 | ERROR    | reliable_finance.main | Критична помилка виконання застосунку: Катего
рія не вказана
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab05>
```

8. Демонстрація принципу Fail-fast: перевірка валідності YAML-конфігурації до початку відкриття файлових ресурсів



9. Верифікація стану Git-репозиторію командою git log --oneline та git status

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git log --oneline -n 5
3237b5d (HEAD -> main, origin/main) add output after running a program
43483f5 Lab05: implement streaming readers, exception chaining, atomic JSON export and ma
in pipeline
931a344 Lab05: implement YAML configuration loader, logging system and context manager
ac3c1ea Lab05: implement custom exception hierarchy and immutable dataclass models
345efa0 Lab05: initial setup, config.yaml, test data and packaging configuration
```

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab05 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Теоретичні відомості та обґрунтування рішень

1. **Ієрархія винятків та Exception chaining:**
    - Створено базовий клас ApplicationError(Exception), від якого наслідуються спеціалізовані помилки предметної області: ConfigurationError, DataImportError, RecordValidationError та DataExportError.
    - Клас RecordValidationError містить атрибути line_number та field, що дозволяє точно локалізувати помилку у вхідному CSV.
    - Застосовано синтаксис raise RecordValidationError(...) from error, що зберігає оригінальну першопричину (наприклад, технічний ValueError при невдалому перетворенні рядка у float) та формує зв'язаний traceback для зручного налагодження.
2. **Принципи проектування EAFP та Fail-fast:**
    - **Fail-fast:** Реалізовано у функції validate_config() під час старту програми. Якщо файл config.yaml відсутній або містить некоректні параметри (наприклад, від'ємне порогове значення суми чи порожній список категорій), програма негайно припиняє виконання ще до спроби відкриття великих файлів із даними.
    - **EAFP (Easier to Ask Forgiveness than Permission):** Використовується під час парсингу полів словника в блоках try/except, де спроба конвертації типу виконується одразу без зайвих попередніх регулярних виразів чи перевірок.
3. **Контекстні менеджери та гарантоване закриття ресурсів:**
    - Усі файлові операції обгорнуто в конструкцію with path.open(...), яка гарантує автоматичний виклик методу close() для дескриптора файлу навіть у разі виникнення непередбаченого винятку.
    - Створено власний генераторний контекстний менеджер @contextmanager def logged_operation(...), який фіксує точний час виконання операції за допомогою perf_counter() та реєструє факт старту/завершення чи збою у файловий лог.
4. **Атомарний запис (Atomic write):**
    - Щоб уникнути пошкодження вихідного JSON-файлу при раптовому збої процесу або знеструмленні, серіалізація здійснюється у тимчасовий файл із розширенням .tmp.
    - Лише після успішного запису останнього символу виконується системний виклик temp_path.replace(path). Якщо виникає помилка

запису, файл .tmp негайно видаляється (unlink), а робочий файл залишається цілісним.

Висновок:

Під час виконання лабораторної роботи було опановано побудову відмовостійких Python-застосунків для імпорту, експорту та обробки структурованих даних. Для предметної області фінансового трекера (Варіант 12) реалізовано потоковий конвеєр, що читає вхідні CSV-файли, валідує параметри відповідно до зовнішньої YAML-конфігурації та здійснює атомарний експорт у формат JSON із використанням контекстних менеджерів.

Створено предметну ієрархію винятків (ApplicationError, ConfigurationError, RecordValidationError, DataExportError) із використанням механізму exception chaining. Практично продемонстровано різницю між політиками обробки помилок: режим Strict забезпечує миттєве переривання при першій невідповідності, тоді як Tolerant гарантує безперервну обробку валідних даних із реєстрацією всіх інцидентів у системному журналі через модуль logging[cite: 6]. Застосунок організовано відповідно до стандарту src-layout, дотримано принципу Fail-fast, а хід виконання зафіксовано в системі контролю версій Git.

Додаток:

Config.py

```
from pathlib import Path
import yaml
from reliable_finance.exceptions import ConfigurationError
from reliable_finance.models import AppConfig, LoggingConfig, ProcessingConfig


def load_config(path: Path) -> AppConfig:
    """Завантаження та сувора валідація YAML конфігурації (Fail-fast)."""
    try:
        with path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"Конфігураційний файл не знайдено: {path}")
from error
    except yaml.YAMLError as error:
        raise ConfigurationError("Некоректний синтаксис файлу YAML") from error


    try:
        config = AppConfig(
            input_path=Path(raw["input"]["path"]),
            output_path=Path(raw["output"]["path"]),
            processing=ProcessingConfig(
                minimum_amount=float(raw["processing"]["minimum_amount"]),
                skip_invalid=bool(raw["processing"]["skip_invalid"]),
                allowed_categories=tuple(raw["processing"]["allowed_categories"
]),
            ),
            logging=LoggingConfig(
                level=str(raw["logging"]["level"]),
                path=Path(raw["logging"]["path"]),
            ),
        )
    except (KeyError, TypeError, ValueError) as error:
```

```
        raise ConfigurationError("Помилка структури YAML: відсутні або
некоректні ключі") from error


    validate_config(config)

    return config


def validate_config(config: AppConfig) -> None:

    if config.processing.minimum_amount < 0:

        raise ConfigurationError("minimum_amount не може бути від'ємною!")

    if not config.input_path.exists():

        raise ConfigurationError(f"Вхідний файл не знайдено:
{config.input_path}")

    if not config.processing.allowed_categories:

        raise ConfigurationError("Список allowed_categories не може бути
порожнім!")
```

Models.py

```
from dataclasses import asdict, dataclass

from pathlib import Path


@dataclass(frozen=True, slots=True)

class ProcessingConfig:

    minimum_amount: float

    skip_invalid: bool

    allowed_categories: tuple[str, ...]


@dataclass(frozen=True, slots=True)

class LoggingConfig:

    level: str

    path: Path


@dataclass(frozen=True, slots=True)

class AppConfig:

    input_path: Path

    output_path: Path
```

```
    processing: ProcessingConfig
    logging: LoggingConfig

@dataclass(frozen=True, slots=True)
class TransactionRecord:
    date: str
    category: str
    amount: float
    op_type: str
    description: str

    def to_dict(self) -> dict[str, object]:
        return asdict(self)
```

Services.py

```
from collections.abc import Iterable, Iterator
from dataclasses import dataclass
import logging
from pathlib import Path
from reliable_finance.exceptions import RecordValidationError
from reliable_finance.models import TransactionRecord
from reliable_finance.readers import read_csv_rows
from reliable_finance.validators import parse_transaction

logger = logging.getLogger("reliable_finance.service")

@dataclass
class ImportStatistics:
    total: int = 0
    valid: int = 0
    invalid: int = 0

def import_transactions(
    path: Path,
```

```
    *,
    skip_invalid: bool,
    allowed_categories: tuple[str, ...],
    statistics: ImportStatistics,
) -> Iterator[TransactionRecord]:
    """Потоковий імпорт операцій із політикою обробки помилок strict або
tolerant."""
    for line_number, row in read_csv_rows(path):
        statistics.total += 1
        try:
            record = parse_transaction(
                row,
                line_number=line_number,
                allowed_categories=allowed_categories,
            )
        except RecordValidationError as error:
            statistics.invalid += 1
            logger.warning(
                "Некоректний запис у рядку %s (поле '%s'): %s",
                error.line_number,
                error.field,
                error,
            )
            if skip_invalid:
                continue
            raise
        statistics.valid += 1
        yield record


def filter_by_minimum_amount(
    records: Iterable[TransactionRecord], minimum_amount: float
) -> Iterator[TransactionRecord]:
    """Фільтрація валідного потоку за пороговою сумою."""
    for record in records:
        if record.amount >= minimum_amount:
```

```
            yield record
```

Validators.py

```
from reliable_finance.exceptions import RecordValidationError
from reliable_finance.models import TransactionRecord


def parse_transaction(
    row: dict[str, str],
    *,
    line_number: int,
    allowed_categories: tuple[str, ...],
) -> TransactionRecord:
    """Парсинг та валідація рядка з ланцюжком винятків (exception chaining)."""
    date_val = row.get("date", "").strip()
    if not date_val:
        raise RecordValidationError("Порожня дата операції",
line_number=line_number, field="date")


    category = row.get("category", "").strip()
    if not category:
        raise RecordValidationError("Категорія не вказана",
line_number=line_number, field="category")


    if category not in allowed_categories:
        raise RecordValidationError(
            f"Категорія '{category}' не входить до переліку дозволених у
конфігурації",
            line_number=line_number,
            field="category",
        )


    try:
        amount = float(row.get("amount", ""))
    except (ValueError, TypeError) as error:
        raise RecordValidationError(
```

```
            "Сума транзакції повинна бути дійсним числом",
            line_number=line_number,
            field="amount",
        ) from error

    if amount <= 0:
        raise RecordValidationError(
            f"Сума має бути строго більшою за 0 (отримано {amount})",
            line_number=line_number,
            field="amount",
        )

    op_type = row.get("type", "").strip().lower()
    if op_type not in ("income", "expense"):
        raise RecordValidationError(
            f"Некоректний тип операції: '{op_type}'. Очікується income або
expense",
            line_number=line_number,
            field="type",
        )

    description = row.get("description", "").strip()

    return TransactionRecord(
        date=date_val,
        category=category,
        amount=amount,
        op_type=op_type,
        description=description,
    )
```

Main.py

```
import logging
from pathlib import Path
from reliable_finance.config import load_config
from reliable_finance.exceptions import ApplicationError
```

```
from reliable_finance.exporters import atomic_export_json
from reliable_finance.logging_config import configure_logging
from reliable_finance.services import (
    ImportStatistics,
    filter_by_minimum_amount,
    import_transactions,
)
from reliable_finance.utils import logged_operation

logger = logging.getLogger("reliable_finance.main")

def main() -> None:
    config_path = Path("config.yaml")

    try:
        config = load_config(config_path)
        configure_logging(config.logging.level, config.logging.path)

        statistics = ImportStatistics()

        with logged_operation("Імпорт, валідація та експорт фінансових
транзакцій"):
            # Потоковий конвеєр
            stream = import_transactions(
                config.input_path,
                skip_invalid=config.processing.skip_invalid,
                allowed_categories=config.processing.allowed_categories,
                statistics=statistics,
            )

            filtered = filter_by_minimum_amount(
                stream,
                config.processing.minimum_amount,
            )
```

```
            exported_count = atomic_export_json(filtered, config.output_path)

        logger.info(
            "Статистика обробки: Всього=%d | Валідних=%d | Некоректних=%d | Експортовано=%d",
            statistics.total,
            statistics.valid,
            statistics.invalid,
            exported_count,
        )

    except ApplicationError as error:
        logger.error("Критична помилка виконання застосунку: %s", error)
        raise SystemExit(1) from error

if __name__ == "__main__":
    main()
```